

Building a Native UI Engine

<https://gemini.google.com/app/8bec89aa9d0d1108>

User prompt: I have ideas for a new markup format for UIs like html/css. I want to explore ways of implementing this that take advantage of graphical hardware acceleration. A couple of ideas come to mind: first, I'm thinking about taking chromium apart so that I could substitute my markup and computational model for that used by html/css. Another idea, would be to simply make a native application (let's say for Mac OSX). If I were to make a native application, I'd still need to understand what graphical libraries I'd be using for my primitives, and surely there's a lot of options. Can you help me with an overview of the options and a comparison?

Response: Building a new markup format and UI layout engine from scratch is a massive systems engineering challenge. You're essentially building a modern browser engine, minus the networking layer and JavaScript execution environment.

Here is a breakdown of your two paths and the graphical libraries you can leverage to implement your own computational model.

Path 1: Taking Chromium Apart

Forking Chromium (specifically the Blink layout engine and Skia rendering pipeline) gives you a working engine, but it is notoriously rigid.

- **The Reality:** Chromium is roughly 35 million lines of C++. Replacing the HTML/CSS parsing and DOM tree with your own markup means decoupling Blink's layout algorithms from its rendering backend.
- **The Verdict:** It's akin to replacing the nervous system of an elephant while it's running. You will likely spend more time fighting Chromium's build system and stripping out legacy web standards than innovating on your core layout ideas.

Path 2: Building a Native Engine

Starting fresh as a native application allows you to build a clean, hermetic architecture. To get your markup onto the screen with hardware acceleration, you need a 2D rendering library that abstracts the GPU (or you must write directly to the GPU using 3D APIs).

Since you are designing a UI, you need an engine that handles 2D vector primitives (paths, bezier curves), text shaping, and compositing.

1. Skia (The Heavyweight Standard)

Skia is the C++ engine that powers Chrome, Android, and Flutter.

- **How it works:** It provides a rich 2D API (canvas, paths, text) and translates it into backend-specific commands for Metal, Vulkan, or OpenGL.
- **The Fit:** It is battle-tested and does practically everything out of the box, including advanced font shaping. However, it is a massive dependency to integrate and can suffer from shader compilation stutter on complex animations.

2. Vello & WGPU (The Modern Rust Approach)

Vello is a next-generation 2D rendering engine built in Rust by the Linebender community.

- **How it works:** It leverages wgpu to compile down to native Metal (or Vulkan) compute shaders. Instead of the CPU calculating vector paths, Vello uses prefix-scan algorithms to offload path sorting, clipping, and rasterization entirely to the GPU.
- **The Fit:** If you are building your engine in Rust, this is the most exciting option right now. It is explicitly designed for multi-threaded scene graph generation, making it incredibly fast for complex, parallelized layouts on multi-core hardware like the M2 Pro.

3. WebRender (The UI Compositor)

WebRender is the Rust-based GPU rendering engine built for Mozilla's Servo browser.

- **How it works:** Unlike Skia (which treats everything as generic 2D shapes), WebRender is optimized specifically for CSS-like UI workloads. It uses the GPU to rapidly composite UI primitives: rounded rectangles, box shadows, borders, and text.
- **The Fit:** If your new markup heavily features box-model semantics rather than arbitrary vector art, WebRender is tailored for your exact use case.

4. Impeller (The Predictable C++ Engine)

Flutter's new rendering engine, built from scratch to replace Skia.

- **How it works:** It pre-compiles a smaller, fixed set of shaders ahead of time rather than generating them dynamically at runtime.
- **The Fit:** If you are using C++ and want guaranteed frame rates without the unpredictable shader compilation latency that plagues older engines, Impeller is a strong, modern choice—though extracting it from Flutter requires some effort.

5. Raw Platform APIs (Metal / WebGPU)

You could build directly on Apple's Metal for a macOS-native app, or use a cross-platform low-level API like WebGPU.

- **The Fit:** Maximum control, zero bloat. But be warned: these are strictly 3D APIs. You would have to write your own algorithms to convert 2D curves into triangles, and writing a modern text renderer (managing glyph atlases, sub-pixel antialiasing, and Unicode shaping) is a multi-year project on its own.

Engine Comparison

Engine	Primary Language	Graphics Backend	Best For	Trade-offs
Skia	C++	Metal, Vulkan, GL	A complete, "batteries-included" 2D pipeline.	Massive binary footprint; legacy architecture.
Vello	Rust	WGPU (Compute)	High-performance, heavily parallelized vector rendering.	Still evolving; newer ecosystem.
WebRender	Rust	OpenGL, Vulkan	UI-specific layouts (box models, shadows, borders).	Less optimized for arbitrary, complex vector art.
Impeller	C++	Metal, Vulkan	Predictable frame times with no shader compilation stutter.	Tightly coupled to the Flutter ecosystem.
Raw Metal	Swift / C++	Metal	Absolute low-level control of the hardware.	You must write your own 2D path rasterizers and font shapers.

User prompt: How's Vello with text?

Response: Vello's approach to text is strictly modular: **Vello does not actually handle text layout or font shaping itself.**

Instead, the Linebender team intentionally decoupled text processing from the core rendering engine. Vello treats text simply as a series of vector paths or cached images to be drawn. To actually get text onto the screen, you use Vello in tandem with its sister crates, which work together as a pipeline.

Here is how the text stack works with Vello as of early 2026:

1. Layout & Shaping: Parley

Parley is Linebender's rich text layout library. If you hand it a string of text, Parley handles:

- Line breaking and text alignment.
- Bidirectional text resolution (bidi) and internationalization, which is now powered by ICU4X.
- Font fallback and computing the exact geometric coordinates for every glyph.

2. Glyph Management: Glifo

Once Parley knows *where* the glyphs go, a newly extracted crate called **Glifo** (moved directly into the Vello repository in Q1 2026) handles the actual font rendering logic.

- It extracts the vector outlines from the font files.
- It manages atlas-based glyph caching (so Vello doesn't have to re-rasterize the same letter thousands of times).
- It handles complex typographic rendering details, like drawing color emojis and calculating ink-skipping for underlines.

3. Rasterization: Vello

Finally, Vello receives the processed glyphs. Because Vello is fundamentally a vector renderer, it takes the paths generated by Glifo and rasterizes them using its high-performance compute shaders (or its newer CPU/Hybrid "Sparse Strips" pipelines). Recent optimizations in Vello have added robust glyph caching for hinting instances and outline paths, making it extremely fast at drawing large amounts of text frame-over-frame.

The Takeaway for Your UI Engine

If you use Vello for your custom markup engine, you won't have to build a text stack from scratch. You can feed your markup's text nodes directly into Parley for layout, use Glifo to extract the shapes, and let Vello blast them onto the GPU. This gives you browser-grade text rendering (including bidi and emojis) without having to write the complex shaping algorithms yourself.

Sources

1. [Linebender in December 2025](#)
2. [Linebender in 2026 Q1 - daily.dev](#)
3. [Linebender in September 2025](#)

User prompt: Can you give me some examples of vello and maybe parley where we tell it render a paragraph of text in a specific font with a certain width and it automatically figure out the layout and can tell me the total number of lines and overall height?

Response: Here is a practical example of how Parley and Vello integrate in the modern Linebender ecosystem.

Because Parley was designed specifically to feed GPU renderers like Vello, their integration is quite elegant. Parley handles all the complex logic (shaping, bidirectional text, line breaking, and metrics), and then yields an iterator of raw glyph IDs and positions that Vello can blast onto the GPU.

Here is how you would set up a paragraph, enforce a maximum width, inspect its dimensions, and draw it:

```
use parley::{FontContext, LayoutContext, Layout, style::StyleProperty};
use vello::Scene;
use vello::peniko::{Brush, Color};
use vello::kurbo::Affine;

// 1. Initialize Contexts
// These are scratch spaces and caches. You should keep these alive
// in your application loop rather than creating them per-paragraph.
let mut font_cx = FontContext::new();
let mut layout_cx = LayoutContext::new();

// 2. Define boundaries and styling
let text = "Building a new layout engine is an ambitious journey. Vello and Parley make the text-rendering as";
let max_width = 350.0; // The width to wrap our paragraph at

// 3. Build the layout tree
// The display scale (1.0) is for DPI scaling.
let mut builder = layout_cx.ranged_builder(&mut font_cx, text, 1.0, true);

// Set default styles for the whole paragraph
builder.push_default(StyleProperty::FontSize(18.0));
builder.push_default(StyleProperty::FontStack(parley::style::FontStack::Source("Inter")));

// Build the un-broken layout
let mut layout: Layout<> = builder.build(text);

// 4. Execute Line Breaking
// This is where Parley actually calculates the line wraps.
layout.break_all_lines(Some(max_width));
layout.align(None, parley::Alignment::Start); // Align left

// 5. Inspect the Results
let num_lines = layout.len();
let overall_height = layout.height();
println!("Layout requires {} lines and is {}px tall", num_lines, overall_height);

// 6. Render with Vello
let mut scene = Scene::new();
let text_color = Color::from_rgb8(200, 200, 200);

// Iterate through the lines and glyph runs inside the layout
for line in layout.lines() {
    for run in line.glyph_runs() {
        // Grab the underlying font data from Parley
        let font_data = run.font().data();

        // Extract the shaped glyph IDs and their exact X/Y coordinates
        let glyphs = run.glyphs().map(|g| vello::glyph::Glyph {
            id: g.id as u32,
            x: g.x,
            y: g.y,
        });

        // Push the glyph run into the Vello scene
        scene
            .draw_glyphs(&font_data)
            .font_size(run.font_size())
            .transform(Affine::translate((0.0, line.metrics().baseline))) // Push line down
            .draw(
                vello::peniko::Fill::NonZero,
                Brush::Solid(text_color),
                glyphs
            );
    }
}
```

Key Architectural Takeaways for Your Engine

1. **Separation of Concerns:** Notice how `Layout` and `Scene` never explicitly reference each other. Parley operates entirely on CPU-side metrics and font tables. It just does the math. Vello has no concept of what a "paragraph" or a "line" is—it only knows how to draw shapes indexed by an ID.
2. **Re-Flowing is Cheap:** Notice `layout.break_all_lines(Some(max_width))`. If the user resizes your application window, you do **not** need to rebuild the layout or re-shape the font. You simply call `break_all_lines` on the existing `layout` with the new width, and Parley re-calculates the line breaks almost instantly.
3. **The Layout Context:** `LayoutContext` is explicitly designed as a reusable memory pool. In a real engine, you will have thousands of text nodes. By passing the same `LayoutContext` to every node you build, Parley recycles the memory allocations for text shaping, preventing massive garbage collection spikes.

If you are replacing HTML/CSS, you will likely map your markup text nodes 1:1 to Parley `Layout` structs, caching them in your UI tree, and only re-calling `.break_all_lines()` when the parent container's box model changes its width.

Sources

1. [Layout in parley::layout - Rust - Docs.rs](#)

User prompt: What is linebreaker?

Response: In the context of our conversation about UI engines, "linebreaker" can refer to two things—one is a core technical component of text rendering, and the other might be a slight mix-up with the **Linebender** organization I mentioned earlier.

Here is a breakdown of both:

1. The Component: What is a "Line Breaker"?

If you are building a custom markup engine, a **line breaker** is the specific algorithmic component responsible for figuring out where a line of text is allowed to wrap when it hits the edge of its container.

While it sounds simple (e.g., "just break the text at the spaces"), it is actually one of the most notoriously complex parts of UI engineering because it has to adhere to the **Unicode Line Breaking Algorithm (UAX #14)**. A proper line breaker has to understand:

- **Soft vs. Hard Breaks:** Hard breaks are explicit `\n` characters. Soft breaks are opportunities to wrap (like spaces or hyphens).
- **CJK Languages:** Chinese, Japanese, and Korean don't use spaces between words, so the line breaker has to know which specific characters are legally allowed to start or end a line.
- **Punctuation Rules:** A line breaker ensures a comma or period never accidentally gets wrapped to the start of a new line.
- **Hyphenation:** Advanced line breakers (using dictionaries) calculate where to inject hyphens into long words if they exceed the container width.

In the Rust ecosystem, this is often handled by crates like `unicode-linebreak` or `icu_segmenter`, which are embedded inside larger layout libraries like Parley or `glyph_brush` (which literally has a `LineBreaker` trait).

2. The Organization: "Linebender"

If you were referring to the name I brought up in the previous response, that is **Linebender** (not linebreaker).

Linebender is an open-source collective of developers (including many who work on Google Fonts and Android's text stack) dedicated to writing next-generation, high-performance 2D graphics and text rendering libraries in Rust. They are the creators of:

- **Vello:** The GPU compute-shader renderer.
- **Parley:** The text shaping and layout engine (which *contains* a line breaker).
- **Xilem:** Their experimental reactive UI framework.

If you are building your own engine from scratch, you will absolutely need a line breaker algorithm in your pipeline. But rather than writing one yourself to parse the massive Unicode rulebook, you would typically feed your text into a layout engine like Parley, which runs the line breaker for you based on the constraints (like `max_width`) of your custom UI elements.

Sources

1. [glyph_brush - Rust - Docs.rs](#)

User prompt: Sorry, I mean linebender. Separately, one of the critical flows for a webbrowser is fast scrolling. How would that work using vello? Could I render a bunch of glyphs and then move my window around?

Response: Fast scrolling is the defining metric for browser performance. In a modern GPU-accelerated engine, you never want to re-calculate layouts or re-rasterize complex vector paths just because the user moved the scroll wheel.

If you are using Vello and WGPU, you have two primary ways to architect scrolling: the "Immediate Viewport" approach (simpler, relies on Vello's raw speed) and the "Compositor Layer" approach (how Chrome and Firefox actually work).

1. The Vello Native Way: Viewport Transforms + Culling

Vello operates on a retained scene graph that you rebuild or modify every frame. However, building the scene graph on the CPU is extremely cheap—it is essentially just writing instructions to a byte buffer.

To scroll, you do not recalculate your text layout. You simply apply a mathematical translation to the scene when you draw it.

1. **Glyph Atlas:** The first time text appears, Glifo rasterizes the individual characters into a GPU texture (the glyph atlas).
2. **Culling:** You maintain the bounding boxes of your layout nodes. If a paragraph's Y-coordinate is completely outside the visible window (the viewport), you skip it. You do not push it into the Vello Scene.
3. **Translation:** For the nodes that *are* visible, you push them into the Vello scene but apply an `Affine::translate(0.0, -scroll_offset_y)` to them.

Because the glyphs are already cached in the GPU atlas, Vello isn't actually re-drawing the complex vector curves of the text as you scroll. It is just taking the pre-rendered letters from the atlas texture and pasting them slightly higher or lower on the screen. This easily hits 120 FPS.

2. The Browser Way: Compositing & Tiled Textures

If your document is incredibly complex (e.g., thousands of overlapping SVG gradients, box shadows, and transformed text), even Vello's compute shaders might drop a frame if you force it to re-rasterize the visible scene every 8 milliseconds.

Modern browsers solve this using **Composited Scrolling**. You can build this on top of Vello using raw WGPU:

1. **Render to Texture:** Instead of rendering the Vello scene directly to the screen (the Swapchain), you render sections of your layout into massive, off-screen WGPU textures (often divided into 256x256 or 512x512 "tiles").
2. **The Compositor Thread:** You separate your engine into a Main Layout Thread and a Compositor Thread.
3. **Texture Shifting:** When the user scrolls, the Compositor Thread takes over. It doesn't ask Vello to render anything. It simply takes those pre-rendered off-screen textures and moves them up using a basic WGPU quad draw.

This is why, when you scroll violently fast on a heavy website, you sometimes see blank white squares for a split second. You scrolled faster than the layout engine could rasterize new tiles, so the compositor just scrolled the empty textures it had available.

How to Implement This in Your Engine

If you are starting out, **do not build a tiled compositor yet**. Start with the Viewport Transform approach.

Since you are building a custom markup engine, your architecture should look like this:

- **Parse Phase:** Parse your custom markup into a DOM-like tree.
- **Layout Phase:** Walk the tree to calculate X/Y widths and heights. Run Parley line-breaking here. Store the final geometry.
- **Render/Scroll Phase (Every Frame):** Read the current `scroll_y` value from the OS. Walk your geometry tree. If `node.y - scroll_y` is on screen, push its Parley glyphs into the `vello::Scene` with that translation applied.

Because Vello's compute architecture handles large numbers of simple overlapping shapes (like text glyphs) efficiently, the Viewport Transform approach will easily carry you through your prototyping phase without needing the architectural complexity of asynchronous texture tiling.

User prompt: Does vello's scene architecture allow me to push everything to the scene, regardless of if it's on or off screen and leave the clipping to it? Will it do unnecessary rendering of off screen curves?

Response: Yes, Vello's architecture allows you to push everything to the scene, and it will automatically handle the clipping. It will **not** rasterize (paint pixels for) off-screen curves or glyphs. However, doing this for a large document introduces a massive bottleneck on the CPU and memory bandwidth.

Here is exactly what happens under the hood if you push an entire off-screen document to the `vello::Scene`:

1. The GPU Side: Tile Binning (The Good News)

Vello is a compute-centric renderer that divides the screen into a grid of tiles (typically 16x16 pixels). When the GPU processes your scene, the first compute shaders it runs handle transformations and calculate the bounding box for every single path, image, or glyph.

It then assigns those objects to the tiles they overlap. If a curve's bounding box is entirely outside the visible render target, the GPU simply doesn't assign it to any tile. Because it is never binned, the heavy rasterization phase completely ignores it. **You do not pay the cost of rendering off-screen pixels.**

2. The CPU Side: The Encoding Tax (The Bad News)

When you call `scene.draw_glyphs()` or `scene.draw_path()`, Vello doesn't immediately talk to the GPU. Instead, it encodes your drawing instructions into a densely packed byte buffer in RAM.

If you have a document that is 50,000 pixels tall, and you push all of it to the scene every frame during a scroll:

1. **CPU Overhead:** Your CPU wastes cycles serializing thousands of off-screen paths and text runs into Vello's encoding buffer.
2. **Memory Transfer:** That unnecessarily massive byte buffer must be transferred across the PCIe bus to the GPU every single frame.
3. **GPU Compute Waste:** Even though Vello skips the rasterization, the GPU's initial compute shaders still have to execute the affine transformations and bounding-box math for every off-screen item just to figure out that they are off-screen.

The Standard Practice: CPU-Side Culling

Because you are building a UI layout engine, you already know the exact bounding box (X, Y, Width, Height) of every text node and `<div>`-equivalent in your document.

You should perform **coarse CPU-side culling** before talking to Vello.

Instead of pushing the whole tree, you walk your layout tree and perform a simple Axis-Aligned Bounding Box (AABB) check:

```
// Only push elements that intersect the viewport (plus a small margin)
let viewport_top = scroll_y;
let viewport_bottom = scroll_y + window_height;

if node.bottom >= viewport_top && node.top <= viewport_bottom {
  // It's visible! Push to Vello scene.
  scene.draw_glyphs(...);
} else {
  // Skip entirely. Vello never knows it exists.
}
```

This hybrid approach leverages what each processor does best: the CPU uses simple arithmetic to instantly discard 95% of the invisible document, and the GPU gets a lean, highly optimized buffer of only the visible elements to push through its compute pipeline.

User prompt: Here's my idea to replace CSS and HTML. The primary feature I want to have is a directed graph of values (heights, widths, positions, etc) where the edges are dependencies each value is expressed as a deterministic function of its inputs. Such a graph should be fast to compute and even faster to update. Also, I think putting the graph construction in the hands of the designer via the markup gives complete control over layout. The hope is to avoid all the pitfalls currently in html/css to position things as you want them. People have to do crazy tricks in order to simply center a div because they are stuck in the various flow and layout models provided. What do you think of the approach?

Response: You are essentially proposing replacing a document-flow engine with a pure Functional Reactive Programming (FRP) Directed Acyclic Graph (DAG). It is a highly robust systems-engineering approach to UI layout.

By forcing layout to be an explicit dependency graph rather than an implicit cascade of rules, you shift the computational model from constraint-solving (which is what browsers do under the hood) to a highly predictable, deterministic state machine.

Here is why this architecture is incredibly powerful, along with the specific engineering traps you will need to design around.

Why the DAG Approach Excels

1. **O(1) or O(log N) Updates:** In the DOM, changing one element's width can trigger a layout recalculation for the entire document (layout thrashing). In a strict DAG, if an animation updates a variable, you only re-evaluate the specific downstream edges. Everything else is safely cached.
2. **Free Parallelization:** Because the dependencies are explicit, you can topologically sort the graph. Any nodes that do not share a dependency path can have their math computed concurrently on separate CPU threads before being handed off to Vello for rendering.
3. **Absolute Determinism:** As you noted, centering a div ceases to be a CSS flexbox mystery. It simply becomes $x = (\text{parent.width} / 2) - (\text{self.width} / 2)$. You are giving the designer mathematical truth rather than stylistic suggestions.

The Traps You Must Solve

To make this viable for real-world UI, your graph compiler will need to handle a few scenarios where strict directed dependencies break down.

1. The fit-content Paradox (Cyclic Dependencies)

The biggest headache in UI engineering is bidirectional reliance.

- A parent container's `height` often depends on the sum of its children's heights.
- But a child's `width` (and therefore its wrapped text height) often depends on the parent container's `width`. If the user's markup wires these dependencies directly, they will create an infinite loop/cycle. Your graph evaluator will need a mechanism to detect cycles and either throw a compile-time error, or implement a multi-pass resolution phase (e.g., calculate widths first, flowing down, then calculate heights, flowing up).

2. The Verbosity of Text Flow HTML's implicit "flow" model is awful for application UI (dashboards, toolbars, modal centering), but it is exceptionally good for its original purpose: text documents. If a designer has to manually write an edge stating `paragraph_2.y = paragraph_1.y + paragraph_1.height + 16px` for an entire blog post, the markup will be unwritable. You will likely need to implement "macro nodes" (like a Stack or Flow node) that automatically generate the underlying sequential dependency edges for their children.

3. Integration with Parley Because text wrapping changes height dynamically based on width, Parley will act as a heavyweight node in your graph. Your DAG will feed a `width` edge into a Parley node, Parley will run the line-breaking algorithm, and it will output a `height` edge that downstream nodes can hook into.

4. Cassowary vs. DAG Apple's AutoLayout uses the Cassowary algorithm, which is similar to your idea but solves *linear equations* (constraints) rather than evaluating a directional graph. Cassowary allows for complex relationships (e.g., "these three buttons must have equal widths, whatever that ends up being"), but it requires a Simplex solver, which is computationally expensive. Your strict DAG will be vastly faster to compute, provided you can give designers an ergonomic way to express those same relationships using only one-way functions.

User prompt: Yes, let's think of a markup syntax. Let's not use html/xml. Instead, I'd prefer something inspired by TeX and/or protocol buffers. Maybe we should start with layout for a sequence of a headers, paragraphs, and maybe margin notes.

Response: Let's combine the strict, typed property binding of Protocol Buffers with the document-friendly content nesting of TeX.

To solve the "verbosity" problem (so you don't have to manually write `y = previous.bottom` for every single paragraph), we can introduce a macro node like `\Flow` that automatically generates those DAG edges for its children, while still allowing explicit overrides.

Here is what the syntax could look like:

```
\Page(id: main, width: 100vw, height: 100vh) {

  // A structural macro that auto-generates sequential Y dependencies
  \Flow(id: article, width: 600, x: 100, y: 50, gap: 16) {

    \Header(id: h1, size: 24) { Reimagining the Document }

    \Paragraph(id: p1) {
      HTML relies on implicit document flow. In our system, this paragraph's
      Y-position is mathematically bound to 'h1.bottom + article.gap'
      by the Flow container.
    }

    \Paragraph(id: p2) {
      Because we use a directed graph, we can break out of this sequence
      trivially by defining explicit dependencies anywhere in the tree.
    }
  }

  // This element exists entirely outside the Flow hierarchy,
  // but establishes a direct DAG edge to a node inside it.
  \MarginNote(
    id: note1,
    width: 200,
    x: article.right + 40,
    y: p2.top
  ) {
    This note is locked to the top of paragraph 2. If the user resizes
    the window and paragraph 1 wraps to a new line, 'p1.height' increases.
    The DAG propagates this down to 'p2.top', which instantly updates
    'note1.y'. No CSS absolute/relative positioning hacks required.
  }
}
```

The Architectural Rules of this Syntax

- Commands define Nodes (`\Node`):** Like TeX, anything starting with `\` is a node in your graph. This tells the parser to instantiate a struct.
- Parentheses define Edges (`(...)`):** Like Protobuf fields, the parentheses contain strictly typed key-value pairs. This is where the DAG equations live. String literals aren't needed unless it's an arbitrary value; otherwise, they are parsed as mathematical references (`article.right`, `p2.top`).
- Braces define Content (`{...}`):** Text is a first-class citizen. Unlike JSON or pure Protobuf where text must be wrapped in `" "`, the `{}` block allows raw text, making it highly ergonomic for authors. The parser treats everything inside `{}` as a `\Text` child node natively.

Why this solves the CSS Margin Note Problem

In HTML/CSS, getting a margin note to track a specific paragraph is famously difficult. If you put the note *inside* the paragraph, it disrupts the text flow. If you put it *outside* the paragraph using `position: absolute`, it loses its awareness of the paragraph's Y-coordinate unless you use Javascript.

In your DAG system, the relationship is just a one-way edge: `margin_note.y = p2.y`.

When your Rust engine (using Parley) calculates the text layout for `p1`, it yields a `height`. The graph marks `p2.top` as dirty, updates it, marks `note1.y` as dirty, updates it, and hands the final coordinates to Vello.

User prompt: Here's my confusion with the syntax above: we have both node types and node ids. But I don't see anything that's implicitly derived from the node type. Maybe I'm not looking hard enough -- surely a Flow containing a Paragraph lays them out vertically in relation to each other. But, how is MarginNote different from a simple text block if everything is explicit in its value formulas? I'm also wondering about how we make relationships between items simply because they're nested (child/parent/sibling relationships). For example, it might be nice to have margin note in the paragraph that it's meant to align with and then use parent.y instead of p2.y. But putting a \MarginNote in \Paragraph causes problems too because now we're mixing text and nodes. Shouldn't it matter where in the paragraph text we put the margin node? Or maybe it doesn't?

Response: You've hit on the exact architectural boundary between the parser, the DAG, and the text engine. You are completely right: if all values are explicit formulas, the "type" name is largely syntactic sugar.

Here is how you resolve the identity crisis of \MarginNote, how nesting creates implicit edges, and how you handle the collision of text and nodes.

1. Primitives vs. Components (The Identity Crisis)

In a pure DAG engine, there are really only two true primitive nodes that the Rust backend cares about:

- \Block: A generic rectangular spatial boundary.
- \Text: A node that hands a string and a max-width to Parley, and returns a height.

Everything else—\Flow, \Paragraph, \MarginNote—is a **Component** (a macro). A \MarginNote is literally just a \Block where the designer has abstracted away the boilerplate.

When the parser reads \MarginNote(y: p2.top), it expands it into the primitive DAG nodes: \Block(width: 200, x: parent.right + 40, y: p2.top)

2. Nesting Creates Scope (The parent and prev edges)

Nesting should not automatically enforce layout (that's the HTML trap), but nesting *does* establish a local coordinate space and a variable scope.

By nesting, the compiler automatically injects implicit edge variables like parent, prev, and next. This eliminates the need to manually assign IDs like p1 and p2 to everything.

```
\Flow(gap: 16) {
  \Paragraph {
    // Structural nesting: The note is a child of the paragraph block.
    // 'parent.top' resolves to this paragraph's Y-coordinate.
    // 'parent.right' resolves to this paragraph's X + Width.
    \MarginNote(x: parent.right + 40, y: parent.top) { ... }

    The actual text of the paragraph goes here...
  }

  \Paragraph {
    // 'prev' automatically references the sibling paragraph above it.
    // This is exactly what \Flow does under the hood for you.
    \MarginNote(x: parent.right + 40, y: prev.bottom) { ... }
  }
}
```

3. The Inline Problem (Mixing Text and Nodes)

You asked: *Should it matter where in the paragraph text we put the margin node?*

Yes, immensely. There is a structural difference between attaching a note to the *top of the paragraph block* versus attaching a note to a *specific word* that might wrap to line 4.

If you put a node *inside* the text flow, your DAG has to interface directly with Parley's line-breaker. Here is how that architecture works:

The Syntax:

```
\Paragraph {
  This is a complex sentence that will wrap across multiple lines.
  Right here \MarginNote(y: self.anchor.y) { This note tracks the word! }
  is where the note belongs.
}
```

The Engine Execution (The DAG + Parley Handshake):

1. **Parse Phase:** The parser sees \MarginNote inline. It strips it out of the text string and replaces it with a Unicode Object Replacement Character (U+FFFC) or a custom Parley inline-box marker.
2. **Layout Pass 1 (Parley):** The DAG feeds the container width to Parley. Parley runs the line-breaking algorithm on the text block.
3. **The Output Edge:** Parley figures out that the Object Replacement Character ended up on Line 3, at pixel y: 45. Parley outputs this coordinate as an explicit property of the text node (e.g., anchor[0].y = 45).

4. **Layout Pass 2 (DAG):** The DAG sees that the `\MarginNote` explicitly bound its Y-coordinate to `self.anchor.y`. The graph propagates the 45 down to the margin note, and Vello draws it there.

By treating inline nodes as anchor points that Parley resolves, you maintain the strict mathematical purity of the DAG. The text engine just becomes a black-box math function: `f(string, width) = [height, anchor_1_x, anchor_1_y]`.

User prompt: Yeah, talk to me about the component definition syntax

Response: To make this engine viable, component definitions must act as pure functions (or macros) that run during the parse phase. They take inputs (properties and nested children) and compile them down into a raw Abstract Syntax Tree (AST) of `\Block` and `\Text` primitives before the DAG math is ever evaluated.

Here is a proposed syntax for defining reusable components, using a `\Def` keyword and a `\Slot` primitive for content injection.

```
// 1. Define the Component and its expected arguments (with optional defaults)
\Def MarginNote(x, y, width: 200) {

  // 2. Expand into primitive nodes
  \Block(
    x: x,
    y: y,
    width: width
  ) {
    // 3. Apply internal styling/formatting nodes
    \Text(size: 12, weight: 600, color: #666) {

      // 4. Inject whatever the user put inside the {...} block
      \Slot
    }
  }
}
```

When a designer writes `\MarginNote(y: p2.top) { Hello }`, the parser instantly desugars it into the primitive AST, filling in the missing defaults.

Solving the \Flow Problem (Generating Edges for N-Children)

If the engine strictly requires explicit math for layout, how does a component like `\Flow` automatically stack an arbitrary number of children without requiring a Turing-complete loop in the markup?

You can solve this by introducing an `\Apply` or `\Cascade` directive. This tells the parser to inject specific DAG equations into all children passed into the slot.

```
\Def Flow(x: parent.x, y: parent.y, gap: 16) {

  \Block(x: x, y: y) {

    // The \Cascade directive attaches these DAG edges to every child node
    // inside the \Slot.
    // 'prev' is a reserved keyword resolving to the sibling immediately above it.
    \Cascade(
      x: parent.left,
      y: prev.bottom + gap // The first child defaults 'prev.bottom' to 'parent.top'
    ) {
      \Slot
    }
  }
}
```

Now, when the user writes:

```
\Flow(gap: 20) {
  \Paragraph { First }
  \Paragraph { Second }
  \Paragraph { Third }
}
```

The parser expands it into the raw, explicit DAG primitives:

```
\Block(x: parent.x, y: parent.y) {
  \Text(x: parent.left, y: parent.top) { First }
  \Text(x: parent.left, y: prev.bottom + 20) { Second }
  \Text(x: parent.left, y: prev.bottom + 20) { Third }
}
```

Architectural Benefits of this Syntax

- **Zero Runtime Overhead:** Because components are just AST macros, they don't exist at runtime. By the time the DAG evaluator starts calculating coordinates for Vello, the tree is completely flattened into raw math edges.
- **Component Encapsulation:** A `\Def` creates a local variable scope. If you define a complex `\Card` component with an internal `image_height = 200` variable, that variable does not leak into the global DAG namespace, preventing edge collisions.
- **Overriding Inherited Math:** Because the DAG is deterministic, explicit declarations always beat inherited ones. If a `\Flow` cascades `x: parent.left` onto its children, but the designer writes `\Paragraph(x: 100) { ... }`, the explicit `100` overwrites the cascaded edge.

User prompt: One thing I want to avoid at all cost are macros. I don't want the compiler to be used as a substitution engine instead of using strict markup or a proper programming/expression syntax. I'm worried slot might function that way.

Response: You are entirely right to be suspicious of macros. When a compiler acts as a naive text-substitution engine (like the C preprocessor or TeX's macro system), you lose type safety, scope encapsulation breaks down, and stack traces become impossible to debug because the runtime engine is executing a completely different structure than what the designer wrote.

If the goal is a strict mathematical Directed Acyclic Graph (DAG), you need a **strongly typed expression syntax**, not an AST-rewriter.

Instead of treating `\MarginNote` as a macro that expands into a block, you treat it as a **Sub-Graph Factory**. It acts as a custom Node type with explicit input ports and strict type contracts.

Here is how you achieve component reuse and nesting using strict programming semantics rather than text substitution.

1. Components as Strongly Typed Sub-Graphs

In a strict syntax, `{ ... }` is not an arbitrary text slot. It is simply syntactic sugar for passing a Node or `Array<Node>` to a specific property (usually called children or content).

You define the component by declaring its exact typed inputs:

```
// The signature defines strict input ports. 'content' expects a Node type.
\Component MarginNote(x: Number, y: Number, content: Node) {

  // The component returns a primitive \Block node.
  \Block(x: x, y: y, width: 200) {

    // Instead of a macro \Slot, we explicitly bind the input variable
    // to the child property of the \Text primitive.
    \Text(size: 12, weight: 600, children: content)
  }
}
```

When the designer uses it in their markup, the parser treats it as a strict function call:

```
// The string "Hello" is parsed as a TextNode and passed into the 'content' port.
\MarginNote(y: p2.top) { "Hello" }
```

There is no macro expansion. The DAG simply instantiates a `MarginNote` node, which internally instantiates a `Block` node. The mathematical edges remain intact and traceable.

2. Solving \Flow with Graph Expressions (No AST Rewriting)

Earlier, I suggested a `\Cascade` directive that magically rewrote children's properties. In a strict expression syntax, that is a macro-like anti-pattern.

If layout is a strict DAG, a `\Flow` component shouldn't rewrite its children. Instead, it should use a proper expression syntax to define edges over an array of nodes. You can achieve this using standard collection mapping operations.

Here is how `\Flow` is defined natively, using a strict `Array` of children and index-based referencing:

```
\Component Flow(x: Number, y: Number, gap: Number, items: Array<Node>) {

  \Block(x: x, y: y) {

    // Use an expression-level map to bind edges.
    // 'index' and 'item' are local loop variables, not macros.
    items.map( item, index) => {

      // The DAG edge is explicitly defined as a ternary expression:
      // If it's the first item, attach to the parent's top.
      // Otherwise, attach to the previous item's bottom.
      item.y = (index == 0) ? parent.top : items[index - 1].bottom + gap;
      item.x = parent.left;
    }
  }
}
```

```

        return item;
    })
}
}

```

The Architectural Result

By using this strict approach, your markup language behaves like a declarative programming language rather than a document preprocessor.

1. **True Type Safety:** The parser knows exactly what a `Number`, `String`, `Node`, or `Array<Node>` is. If a designer tries to pass a string into a port expecting an array of blocks, it fails at compile time with a clear error.
2. **Traceable Edges:** If a value is wrong on screen, the DAG debugger can trace the edge from `items[2].y` directly back to `items[1].bottom + gap`. With macros, that traceability is lost the moment the AST expands.
3. **No Hidden Scope:** Variables inside a `\Component` (like `index` or `gap`) are strictly scoped to that sub-graph. They cannot accidentally overwrite variables in the parent scope, which is the primary failure mode of text-substitution engines.

User prompt: I don't think there's a clear distinction between `items.map` as text or code. Also, this seems to introduce two different levels of programming/expression evaluation. One to setup the graph, coming from the definition of components, and another in the definitions of the values themselves. I look at `"item.x = parent.left"` and wonder about whether there's early or late binding of the value of `parent.left`.

Response: You have successfully diagnosed the exact architectural flaw in React, Vue, and SwiftUI: they are all "programs that write programs."

You are entirely right to push back on `items.map`. The moment we introduce closures, loops, and iterators to generate the UI, we lose the strict, single-phase markup model. You end up with a "Generation Phase" (executing the `map` code to build the tree) and a "Layout Phase" (the engine actually calculating pixels).

If we want a mathematically pure markup language, we must eliminate the generation phase. Here is how we resolve both the binding confusion and the array-layout problem using pure, single-phase algebra.

1. Early vs. Late Binding: It is an Equation, not an Assignment

In a traditional programming language, `x = parent.left` is early-bound imperative assignment. It asks, "What is `parent.left` *right now*?" and assigns that number to `x`.

In your DAG markup, the `=` operator must represent **late-bound algebraic identity**.

When the parser reads `x = parent.left`, it does not evaluate any numbers. It simply draws a directed edge in memory from the `parent.left` port to the `x` port.

- **Parse Phase:** Construct the graph of empty nodes and wire the edges.
- **Evaluation Phase:** Data flows through the edges. When Parley finishes shaping a font, it outputs a height, which flows down the edges to resolve the numbers sequentially.

By enforcing late binding everywhere, the graph remains completely reactive. If the window resizes, we don't re-parse or rebuild the graph; we just push new values through the existing edges.

2. Replacing Code with Recurrence Relations

If we strip out procedural code like `items.map`, how do we wire an array of 50 paragraphs together?

The answer lies in how mathematicians define sequences without writing loops: **Recurrence Relations**. We can define the DAG edges for an array by declaring a base case (the first item) and a recursive rule for the *n*th item.

Here is what the pure, loop-free markup looks like for the `\Flow` component:

```

\Component Flow(x: Number, y: Number, gap: Number, items: Sequence<Node>) {
    \Block(x: x, y: y) {
        // 1. The Base Case: Wire the first item in the sequence
        items[0].top = parent.top

        // 2. The Recurrence Relation: Wire the 'nth' item to the 'n-1' item
        items[n].top = items[n-1].bottom + gap

        // 3. The Universal Rule: Wire all items identically on the X-axis
        items[*].left = parent.left

        // Finally, inject the wired sequence into the block
        children: items
    }
}

```

Why this is a Breakthrough for UI Engines

This completely eliminates the "two levels of programming" you were worried about.

1. **No Loops, No Closures:** The parser doesn't execute anything. When it sees `items[n].top = items[n-1].bottom`, it natively understands that this is a topological chain. It creates those edges internally for however many items happen to be in the Sequence array.
2. **True Declarative Markup:** This reads like TeX or math notation, not Javascript. You are stating the absolute geometric truth of the relationship, rather than writing a script to position things manually.
3. **Ergonomic Usage:** The designer utilizing this component never has to think about the DAG math. They just write:

```
\Flow(gap: 16) {
  \Paragraph { First block }
  \Paragraph { Second block }
  \MarginNote(y: items[1].top) { Note attached to the second block! }
}
```

Notice how the `\MarginNote` can easily reach into the Flow's sequence and bind its `y` axis to `items[1].top`. Because it's all just late-bound algebraic edges, the DAG engine guarantees that the note will track that specific paragraph perfectly, without ever requiring a layout recalculation or breaking the component encapsulation.

User prompt: I'm not strictly opposed to two phase execution. Any modern UI needs to have a scripting language that is capable to altering the dom. So, DOM updates need to happen -- and layout calculations need to happen. I do like a declarative style, including recursion and the like. I was wondering if this could be solved with some kind of templating approach. Every paragraph in a flow is easily defined in terms of "prev" referring to the previous sibling. Obviously this causes ergonomic issues if you want to have some mixture of types. It makes me wonder if the definition of Flow should simply push down a definition or template to its children.

Response: Accepting a two-phase architecture—where a scripting layer constructs and mutates the tree, and a reactive DAG strictly evaluates the layout—is the most pragmatic path. It mimics the successful separation of concerns seen in game engines (gameplay scripts vs. the renderer).

Your instinct to have Flow "push down" a template perfectly solves the ergonomic problem of mixed types. In UI architecture, this is the distinction between **Intrinsic Layout** (how big an element is based on its content) and **Extrinsic Layout** (where an element is placed based on its container).

If a `\Paragraph` or `\Header` has to explicitly reference `prev` in its own definition, you are bleeding extrinsic layout into an intrinsic component. By using a templating approach, the container assumes total responsibility for the wiring.

Here is how a push-down template looks using a declarative mapping syntax:

```
// 1. Intrinsic Components (Oblivious to their surroundings)
\Component Header(size: Number, text: String) {
  \Text(font: "Inter", weight: 700, size: size) { text }
}

\Component Paragraph(text: String) {
  \Text(font: "Georgia", size: 16) { text }
}

// 2. Extrinsic Container (Pushes the layout template down)
\Component Flow(x: Number, y: Number, gap: Number, items: Sequence<Node>) {

  \Block(x: x, y: y) {

    // The template wraps every child regardless of its type.
    // The engine automatically binds the 'prev' keyword to the
    // previously yielded template block.
    \Template(sequence: items) { child =>

      \Block(
        x: parent.left,
        y: (child.isFirst) ? parent.top : prev.bottom + gap
      ) {
        // The mixed-type node (Header, Paragraph, etc.) is injected here
        child
      }

    }
  }
}
```

Why this Architecture Scales

1. **Polymorphism is Free:** The Flow component doesn't care if `child` is a `\Paragraph`, an `\Image`, or an entirely nested `\Flow`. It just wraps the child in a structural `\Block` that establishes the `prev.bottom` DAG edge. The child only needs to compute its own internal height.

2. **Clean DOM Mutation:** If your scripting language (the first execution phase) inserts a new `\Header` between two existing paragraphs, the engine simply inserts the new node into the `Sequence`. The `\Template` automatically re-evaluates, generating the new `\Block` wrapper and hot-wiring the DAG edges (`prev.bottom`) to the surrounding nodes. The DAG mathematically settles the new positions in milliseconds.
3. **No Phantom Edges:** Because the wrapper blocks are physically instantiated in the tree by the template, your graph evaluator doesn't have to guess what `prev` means. `prev` just becomes a pointer to the literal `\Block` node instantiated directly before it in the DOM.

The Scripting Handoff

By separating these concerns, your two phases have strict boundaries:

- **Phase 1 (The Script/Template):** A lightweight procedural language (like TypeScript or a custom WASM-compiled language) handles state, data fetching, and executing templates like `\Template(sequence: items)`. It outputs a static tree of wired DAG nodes.
- **Phase 2 (The Graph Engine):** Your Rust/Vello engine takes that static tree, feeds the text into Parley, resolves all the mathematical edges, and blasts the pixels to the GPU.

User prompt: Let me interrogate the implementation here. First, Flow is a block. Next, the template wraps children in blocks. Why are these necessary? Shouldn't we be able to mutate the definition of the children's values directly using the template? Further, instead of `child.isFirst`, maybe simply "`prev ? prev.bottom + gap : parent.top`".

Response: You are absolutely right. The extra `\Block` wrappers were a failure of imagination on my part—I was dragging the HTML DOM "div soup" mentality into a DAG engine where it doesn't belong.

If we have a strict two-phase execution, the template should absolutely just rewrite the child's input edges directly. Wrapping children in invisible blocks bloats the graph memory, slows down traversal, and deepens the tree for Vello without adding any visual value.

Your `prev ? ... ternary` is also structurally superior. Instead of tracking loop state (`isFirst`), it simply asks the graph: *does a preceding edge exist?*

Here is the much cleaner, flattened implementation based on your interrogations:

```
\Component Flow(x: Number, y: Number, gap: Number, items: Sequence<Node>) {
  // The Flow itself provides the bounding box / coordinate space.
  \Block(x: x, y: y) {
    \Template(sequence: items) { child =>
      // Directly mutate the child's ports before yielding it to the DAG.
      // No wrapper block required.
      child.x = parent.left;
      child.y = prev ? prev.bottom + gap : parent.top;

      yield child;
    }
  }
}
```

Why this is Architecturally Superior

1. **A Flatter DAG:** Vello and your Rust backend want the flattest list of primitives possible. By mutating `child.x` and `child.y` directly, the `\Header` or `\Paragraph` nodes become direct children of the Flow's bounding `\Block`. The math resolves in one step instead of bouncing through an intermediate wrapper.
2. **Pure Functional Evaluation:** `prev ? prev.bottom + gap : parent.top` perfectly models a directed graph edge check. At execution time, the engine looks at the `prev` pointer. If it's null, the edge routes to `parent.top`. It's mathematically sound and avoids internal state variables.
3. **True Separation of Layout and Content:** The author of `\Header` never has to think about X/Y coordinates. The `\Header` component only defines its intrinsic properties (font size, weight). The `\Template` acts as the extrinsic layout authority, wiring the coordinate ports on the fly.

User prompt: Why is this functional instead of declarative? I don't see why these instructions need to be computed in order, for example.

Response: You are completely right. I fell back into the exact trap we were trying to avoid.

Writing `child.x = ...` followed by `child.y = ...` and `yield` is fundamentally imperative. It implies a sequential, step-by-step execution context—a script running inside a closure—rather than a pure statement of truth.

If the system is truly declarative, the order of definitions shouldn't exist, let alone matter. The compiler should just look at a set of relationships and wire the graph simultaneously.

To make this strictly declarative and order-independent, we need to eliminate the lambda closure (`child =>`), the mutation assignments (`=`), and the `yield`. Instead of a procedural map function, the template should just be a declarative binding block.

Here is what a purely declarative, order-independent syntax looks like:

```
\Component Flow(x: Number, y: Number, gap: Number, items: Sequence<Node>) {
  \Block(x: x, y: y) {
    // A purely declarative binding. No closures, no execution steps.
    \Apply(sequence: items) {
      x: parent.left,
      y: prev ? prev.bottom + gap : parent.top
    }
  }
}
```

Why this is truly declarative

1. **No Execution Order:** Inside the `\Apply` block, `x` and `y` are just key-value pairs in a data structure. The parser doesn't "run" `x` and then "run" `y`. It simply reads the block and adds those two edges to every node in the sequence.
2. **No Phantom State:** There is no `child` variable to mutate and no `yield` to return it. We aren't creating a temporary execution scope. We are statically declaring that "for all nodes in this sequence, these property relationships hold true."
3. **Graph Compiler friendly:** Because there are no sequential instructions to evaluate, your Phase 1 script (the tree builder) is just generating JSON-like AST nodes. The Phase 2 DAG engine consumes this declarative map and wires the dependencies in whatever order is mathematically optimal.

This keeps the engine strictly in the realm of algebra. The `\Apply` node isn't a function that runs; it's just a routing rule for the DAG.

User prompt: Explain the "`\Apply(sequence: items)`" directive to me. Does `items` refer to the items passed to the flow? What does `sequence` refer to? Why couldn't we simply say `\Children{ x: parent.left, y: prev ? prev.bottom + gap : parent.top }`?

Response: You are completely right, and your intuition here is pointing straight at a much better API design. `\Children` is a massive improvement.

To answer your specific questions about the old syntax before we throw it away:

- `items` referred to the specific variable `items` passed into the `Flow` component's signature (`items: Sequence<Node>`).
- `sequence` was just the named parameter port on the `\Apply` primitive (e.g., `target_list = items`).

It was overly verbose because it was treating the children as an arbitrary variable being passed around, rather than acknowledging the fundamental structure of a UI tree.

Why `\Children` is the correct approach

By introducing a `\Children` directive, you completely eliminate the need to pass lists of nodes around as variables. It acknowledges that nesting is a first-class concept in this language.

If we assume all components implicitly accept nested content (like HTML elements or React's `props.children`), the component definition becomes incredibly clean. You don't even need `items` in the signature anymore:

```
// We drop 'items' from the signature entirely.
\Component Flow(x: Number, y: Number, gap: Number) {
  \Block(x: x, y: y) {
    // \Children acts as a declarative proxy.
    // It says: "Whatever is nested inside this component,
    // inject these DAG edges into them."
    \Children {
      x: parent.left,
      y: prev ? prev.bottom + gap : parent.top
    }
  }
}
```

How the Compiler Handles `\Children`

Under the hood, `\Children` acts as a structural spread operator during the parsing phase. It is not a node itself; it is an instruction to the graph compiler.

When a designer writes:

```
\Flow(gap: 16) {
  \Paragraph { A }
  \Paragraph { B }
}
```

The parser expands the Flow component, hits the \Children directive, and effectively does this:

1. Grabs the nested nodes (Paragraph A, Paragraph B).
2. Iterates through them and attaches the mathematical edges you defined inside the \Children block (`x: parent.left, y: prev...`).
3. Inserts them directly into the parent \Block.

The Architectural Win

This perfectly achieves your goal of a strict, declarative expression syntax:

1. **No Variables to Track:** Designers don't have to wire array variables like `items` to directives like `\Apply`. The physical nesting of the markup is the data structure.
2. **True Extrinsic Layout:** The Flow component acts as a pure layout container. It doesn't care what the children are, and it doesn't need to bind them to a variable. It just defines the rules for its internal coordinate space, and the \Children proxy applies them.

User prompt: Yes, property collisions is important to talk about here. Because now I'm defining values on children that might not be there by default and might not be utilized in the graph. There's also a scope issue -- what if the values are meant to be internal to the child, rather than part of the external interface?

Response: This touches on the most critical safeguard in DAG-based systems: **graph integrity and encapsulation**. If you treat nodes as hardware chips with explicit input ports, both scope and collision problems solve themselves cleanly at compile time.

Here is how the architecture handles scope, missing properties, and collisions without breaking the strict mathematical graph.

1. Scope: The "Hardware Chip" Model

A component is a black box. The only way the outside world (like a \Flow parent) can interact with a child is through the ports explicitly defined in that child's signature.

```
// The signature exposes exactly three public ports: width, text, and color.
\Component Paragraph(width: Number, text: String, color: Color) {

  // 'line_height' is an internal, private DAG variable.
  // It cannot be targeted or overwritten from the outside.
  let line_height = width > 500 ? 24 : 16;

  \Block(width: width) {
    \Text(color: color, line_height: line_height) { text }
  }
}
```

If a parent container tries to use `\Children { line_height: 30 }`, the compiler throws a strict type error: `Error: Component Paragraph does not expose public port 'line_height'`. The parent cannot pierce the encapsulation boundary to mutate the child's internal graph.

2. The Base Spatial Trait (Handling Unutilized Properties)

You asked: *What if the parent defines values the child might not utilize?*

In a UI layout engine, every component that takes up physical space must implement a universal base interface (a Trait). Whether it is a \Paragraph, a \Video, or a \Button, it inherits standard spatial input ports: `x`, `y`, `width`, `height`.

When \Flow uses `\Children { x: parent.left }`, it is guaranteed to work because the compiler knows every visual node possesses an `x` port. If you try to pass an arbitrary, non-standard property in a \Children block (e.g., `\Children { icon_size: 24 }`), the compiler rejects it unless every nested child explicitly exposes an `icon_size` port.

3. Resolving Collisions: The Precedence Rule

If a DAG has two edges trying to drive the exact same input port, it causes a short circuit. The compiler must resolve it.

The Scenario:

```
\Flow {
  // \Children dictates that all children get `x: parent.left`

  // But this paragraph explicitly demands `x: 200`
```

```
\Paragraph(x: 200) { ... }
}
```

The Resolution (Explicit Overrides Ambient): Because the graph is constructed during the parse phase, the AST handles the collision before the math ever executes. The rule is simple: **Direct instantiation wins**.

When the parser processes the `\Flow` block:

1. It looks at the `\Children` routing rules.
2. It looks at the explicitly wired ports on the child `\Paragraph(x: 200)`.
3. It sees the collision on the `x` port.
4. It discards the ambient edge (`parent.left`) for that specific port, and wires the explicit edge (`200`). The `y` edge from `\Children` remains intact.

Advanced Edge Control: `!Force` vs `?Default`

If you want to give component authors absolute control over this collision behavior, you can introduce syntax modifiers to the `\Children` block.

- **Defaults (?)**: "Apply this edge only if the child didn't wire it." (This is the standard behavior described above).
- **Forced (!)**: "I am a strict layout container like a Grid. I dictate the X/Y coordinates. If the user tries to override this, throw a compile error."

```
\Component StrictGrid(...) {
  \Block(...) {
    \Children {
      // The child is forbidden from overriding its coordinates.
      // The Grid's math is absolute.
      x!: calculate_column(index),
      y!: calculate_row(index),

      // The grid suggests a width, but the child can explicitly override it.
      width?: column_width
    }
  }
}
```

This model preserves the pure declarative nature of the system. The compiler never guesses what a property means. It checks the component's public ports, routes the explicit wires first, fills in the ambient wires second, and rejects anything that violates encapsulation or type safety.

User prompt: That seems fine for now. I like the idea of requiring the children to present values used by the container. Let's talk about the the direction of the DAG compared to the parent-child relationship. There are surely times where values are sent top-down to children. We can see that with the `x` value of the children in a flow. There should also be examples of bottom up. For instance, what if we have fixed width children and we want to define the width of their contain to be the max of those of the children.

Response: This is the exact inflection point where traditional HTML/CSS becomes a nightmare (the `fit-content` vs `100%` width struggle), but where a pure DAG absolutely shines.

In UI architecture, this bidirectional data flow maps to two specific concepts:

- **Extrinsic Layout (Top-Down)**: The container tells the child how big it is.
- **Intrinsic Layout (Bottom-Up)**: The child tells the container how big it needs to be to hold its content.

Because your engine is a pure graph, it doesn't care whether the edges point up or down the DOM tree. An edge is just an edge. To support bottom-up evaluation, we just need standard mathematical **aggregation functions**.

Here is how a bottom-up `ShrinkWrapBox` container works in your declarative syntax.

The Bottom-Up Syntax

Instead of routing values *down* through the `\Children` directive, the parent `\Block` pulls values *up* by querying the `children` array directly using standard collection math (e.g., `max`, `sum`).

```
\Component ShrinkWrapBox(padding: 16) {
  \Block(
    // Bottom-Up: Parent explicitly binds its dimensions to the children's outputs
    width: max(children.width) + (padding * 2),
    height: sum(children.height) + (padding * 2)
  ) {
    \Children {
```

```

    // Top-Down: Parent still dictates the children's X/Y coordinates
    x: parent.left + padding,
    y: prev ? prev.bottom : parent.top + padding
  }
}
}

```

When a designer uses this:

```

\ShrinkWrapBox {
  \Button(width: 100) { "Cancel" }
  \Button(width: 250) { "Confirm Action" }
}

```

How the DAG Evaluates It (The Topological Sort)

When the graph compiler parses this, it ignores the DOM tree hierarchy and strictly looks at the mathematical dependencies. It builds this explicit edge list:

1. Button_1.width = 100
2. Button_2.width = 250
3. ShrinkWrapBox.width = max(Button_1.width, Button_2.width) + 32
4. Button_1.x = ShrinkWrapBox.left + 16
5. Button_2.x = ShrinkWrapBox.left + 16

Before evaluating a single number, the graph engine runs a standard **Topological Sort** (like Kahn's Algorithm). This orders the nodes so that every variable is computed *after* its dependencies.

The engine executes in this order:

1. It trivially evaluates the buttons' widths (100, 250).
2. It flows those values *up* to evaluate the Box width (250 + 32 = 282).
3. Now that the Box has a width (and thus a left coordinate), it flows that value *down* to position the buttons on the X-axis.

The parent-child DOM relationship is completely irrelevant to the math. The evaluation order is driven purely by the data dependencies.

The "Fit-Content" Paradox (Handling Cycles)

This architecture gives you an incredible superpower that CSS lacks: **provable cycle detection**.

In CSS, if you tell a parent `width: min-content` and its child `width: 100%`, the browser's layout engine has to guess what you meant, often resulting in layout thrashing, multi-pass rendering, or weird visual popping.

In your DAG engine, this is mathematically impossible. If a designer writes:

```

\Component ParadoxBox() {
  \Block(
    width: max(children.width) // Parent width depends on child width
  ) {
    \Children {
      width: parent.width      // Child width depends on parent width
    }
  }
}

```

During the Parse Phase, the Topological Sort will instantly hit a circular dependency. The compiler simply throws a fatal error: Error: Cyclic dependency detected. ParadoxBox.width relies on Child.width, which relies on ParadoxBox.width.

This is a massive feature. It forces the UI engineer to explicitly decide who owns the truth for that specific axis—the container or the content—preventing an entire class of layout bugs before the application even compiles.

User prompt: Let's talk about z-index and how it should relate (or not relate) to the nesting of children. Here's a scenario: Imagine I want a paragraph on top of a shaded box. The text needs to be in the foreground and the box needs to be in the background. I think typically languages would do this by nesting the paragraph within a box construct. In the examples you've written, you keep having a \Component containing a \Block containing a \Children template change. In my mind, the \Block isn't needed if there's no background needed for the children. The layout information is entirely within the children. But when it comes time to add a background, I'm wondering if the Block needs to wrap the children or not. I'm not sure what the right way to put the child parent relationship. I'm trying to think about how to control this all with value calculations and limit the number of implicit rules. But maybe the z-index should increase as we go to children? Another option is to push the background as a block into the children. Then it's z-index would simply be parent.z-index - 1.

Response: You have just hit on one of the most profound architectural realizations in UI engine design: **layout is not painting**.

In HTML/CSS, the `<div>` is fundamentally flawed because it conflates two completely different jobs. It is both a *layout boundary* (grouping children) and a *paint primitive* (drawing a background and border). This forced coupling is exactly what creates CSS stacking context nightmares and "z-index wars."

Your instinct is absolutely correct: a pure layout container does not need a visual `\Block` wrapper. If it doesn't draw pixels, it shouldn't exist in the render tree.

Here is how you separate the two using your pure DAG approach, validating your idea of pushing the background in as a mathematical sibling.

1. The Separation of `\Scope` and `\Rect`

We need to introduce a new primitive: `\Scope` (or `\Group`). A `\Scope` draws absolutely nothing. It is purely a mathematical namespace in the DAG. It holds X, Y, Width, Height, and Z values for its children to reference, but Vello never sees it.

When you want a background, you do exactly what you suggested: you instantiate a paint primitive (`\Rect`) alongside the content, and use explicit math to handle the layering.

```
\Component ShadedBox(x: Number, y: Number, width: Number, height: Number, bg_color: Color) {
    // 1. The Layout Boundary (Invisible math namespace)
    \Scope(x: x, y: y, width: width, height: height, z: 0) {

        // 2. The Background (A visual primitive)
        // It explicitly binds its dimensions to the parent Scope.
        // As you suggested, it pushes its Z-index down mathematically.
        \Rect(
            x: parent.x,
            y: parent.y,
            width: parent.width,
            height: parent.height,
            color: bg_color,
            z: parent.z - 1
        )

        // 3. The Content
        // The children are technically siblings to the \Rect in the DAG.
        \Children {
            z: parent.z + 1
        }
    }
}
```

Why this is Architecturally Perfect

- 1. No Implicit Stacking Contexts:** You have completely eliminated the HTML rule that "a child can never escape its parent's z-index." Because z is just a mathematical edge, a designer could explicitly pass a child z: 9999 and it would mathematically resolve to be on top of everything on the screen, regardless of how deep in the tree it is nested.
- 2. Zero Render Overhead:** If a designer uses `\ShadedBox` but passes `bg_color: transparent` (or omits the background), your component could just omit the `\Rect` entirely. The layout remains identical, but Vello has one less shape to process.
- 3. True Composition:** A background isn't a property; it's a shape. This means if a designer wants a background that is a circle, or an image, or a video, they don't have to wait for the language authors to add `background-image` or `border-radius` to a `\Block`. They just drop an `\Image` primitive into the scope behind the children.

The "Painter's Algorithm" Alternative (Array Order)

While calculating explicit `z: parent.z + 1` edges is mathematically pure, hardware-accelerated 2D pipelines (like Vello, Skia, and WebRender) are specifically optimized for the **Painter's Algorithm**.

The GPU assumes that whatever is handed to it *last* in the array should be painted *on top* of what came before it.

If you want to reduce the verbosity of calculating explicit Z-index edges everywhere, you can establish one implicit rule: **Execution order defines visual layering (unless explicitly overridden)**.

Because your component definitions are static templates, the order in which they appear in the AST dictates their base layering:

```
\Component ShadedBox(...) {
    \Scope(...) {

        // Array Index 0: Painted first (Background)
        \Rect(...)

        // Array Index 1: Painted second (Foreground)
```

```

    \Children
  }
}

```

If you rely on array order for standard layering, you only need to expose explicit *z-index* DAG edges for the rare exceptions where a designer actually needs to break the physical flow (e.g., a sticky header, a modal, or a tooltip).

This gives you the best of both worlds: the engine remains a flat list of draw commands for Vello, but the designer has absolute, mathematical control over the Z-axis when they need it, without fighting implicit DOM boundaries.

User prompt: I like what you're doing with the scope. However, I think it should be implicit that each component defines a scope. So, in your ShadedBox example, we'd have one less level of nesting.

Response: Baking the scope directly into the component definition is a massive ergonomic win. It removes the boilerplate `\Scope` wrapper and treats the `\Component` block itself as the lexical environment and mathematical namespace.

By enforcing that **every component intrinsically defines a layout scope**, you solidify the rule that components are the primary boundary for data flow and encapsulation.

Here is what the flattened architecture looks like.

The Flatter, Implicit-Scope Syntax

Assuming every component automatically inherits the Base Spatial Trait (*x*, *y*, *width*, *height*, *z*), those variables are implicitly available as input ports in the component's local namespace.

```

\Component ShadedBox(bg_color: Color) {

  // 1. The Paint Primitive (Background)
  // It directly binds to the component's implicit spatial input ports.
  // Because it is defined first, it sits at the bottom of the Painter's stack.
  \Rect(
    x: x,
    y: y,
    width: width,
    height: height,
    color: bg_color
  )

  // 2. The Content
  // Defined second, it naturally paints on top of the Rect.
  \Children {
    // We only explicitly wire what needs to change from the component's base.
    // For example, adding padding:
    x: x + 16,
    y: y + 16
  }
}

```

Why this is the Optimal Architecture

- 1. Perfect Encapsulation:** The variables *x*, *width*, and *z* inside the component body refer strictly to *this specific instance* of `ShadedBox`. The designer doesn't have to write `self.x` or `parent.x`. The component acts as a pure function: it receives its boundaries from the outside, and internally maps those boundaries to its primitives.
- 2. Zero-Cost Abstractions:** Because the component is just a namespace, the parser flattens this immediately. When a designer writes `\ShadedBox(x: 100, y: 200, bg_color: #333)`, the DAG compiler instantly translates it into a `\Rect` at *x: 100* and the children at *x: 116*. The `ShadedBox` itself vanishes from the runtime graph.
- 3. No Z-Index Wars:** Relying on the Painter's Algorithm (array order) within the component scope eliminates the need for explicit *z*: *z - 1* math 99% of the time. The background is declared before the children, so Vello draws it first. If a child needs to break out, it can still explicitly request a higher *z*, but the default behavior requires zero mental overhead.

Handling the "Fit-Content" Paradox in this Model

Because the component itself is the scope, bottom-up (intrinsic) sizing becomes incredibly elegant. You can define default dimensions in the component signature that execute aggregation functions if the consumer doesn't explicitly pass a width or height.

```

// If the consumer doesn't provide a width, it defaults to wrapping the children.
\Component ShadedBox(bg_color: Color, width: max(children.width) + 32) {

  \Rect(
    x: x,
    y: y,

```

```

        width: width, // Safely uses whatever 'width' resolved to in the signature
        height: height,
        color: bg_color
    )

    \Children {
        x: x + 16,
        y: y + 16
    }
}

```

This model is extremely robust. The component exposes a clear contract, acts as a strict namespace, and compiles down to nothing but raw drawing primitives and topological edges for the renderer.

User prompt: How do I take this discussion and bring it into antigravity to have it start building it? I'd love some documents describing the language and its semantics.

Response: Here is a comprehensive architectural and semantic specification document. You can save this as SPEC.md or ARCHITECTURE.md and feed it directly into Antigravity (or any AI coding agent/workspace) to give it the exact constraints, mental models, and syntax required to start building the parser and graph compiler.

This document is written explicitly to prevent an AI from falling back into HTML/CSS or React DOM mentalities.

DAG-UI: A Pure Functional Reactive Layout Engine

Version: Draft 1.0

Target Backend: Rust (Graph Compiler), Vello (GPU Compositor), Parley (Text Shaping)

1. Core Philosophy

DAG-UI replaces the HTML/CSS document-flow model with a strict **Directed Acyclic Graph (DAG)** of layout variables. **Layout is Math:** X, Y, Width, Height, and Z-index are not implicit rules; they are explicit, late-bound equations. **Layout is not Painting:** Grouping elements does not draw pixels. Visuals (like backgrounds) are strictly deferred. **Two-Phase Execution:**

- Phase 1 (Template/Scripting):** A standard scripting language constructs the AST and wires the declarative layout equations.
- Phase 2 (Graph Evaluation):** A Rust engine topologically sorts the equations (Kahn's Algorithm), resolving dependencies.

2. Execution Model & The Graph

There is no "DOM Tree" at runtime. The runtime is a flat graph of variables.

Late-Bound Edges

The ``=`` operator does **not** perform imperative assignment. It establishes a directed edge in the graph. ``width = parent.width / 2`` means: "Draw an edge from ``parent.width`` to this node's ``width`` port." The math is resolved at evaluation time.

Cycle Detection

Because layout relies on a topological sort, bidirectional dependencies (e.g., Parent width relies on Child width) are strictly forbidden.

3. Language Semantics

3.1 Components & Implicit Scope

A ``\Component`` is a black box. It acts as a pure function and an implicit mathematical namespace. **Every component implicitly inherits the Base Spatial Trait** (``x``, ``y``, ``width``, ``height``, ``z``). These spatial variables are natively available in the component's internal scope. Component ports are strictly typed.

```

`` `text
\Component ShadedBox(bg_color: Color, width: max(children.width) + 32) {

    // Paint Primitive: Uses the component's implicit spatial variables
    \Rect(
        x: x,
        y: y,
        width: width,
        height: height,
        color: bg_color
    )

    // Content Integration
    \Children {
        x: x + 16,
        y: y + 16
    }
}

```

3.2 The \Children Directive

`\Children` is not a node; it is a structural spread operator evaluated at compile time. It takes the nested nodes provided by the consumer and explicitly applies a set of mathematical edges to them.

3.3 The `prev` Keyword

To define sequences without procedural loops, DAG-UI relies on recurrence relations. The `prev` keyword allows a node injected via `\Children` to reference the output ports of the node immediately preceding it in the AST array.

- Example: `y: prev ? prev.bottom + gap : top`

3.4 Layering (The Painter's Algorithm)

Z-index wars are eliminated by separating grouping from painting.

- **Default:** Execution array order defines layering. Primitives declared later in a component paint on top of earlier ones.
- **Override:** Any node can explicitly declare a z edge (e.g., `z: parent.z + 100`) to mathematically break out of the Painter's Algorithm.

4. Syntax & Grammar Reference

Instantiation: Nodes begin with `\` followed by their Component or Primitive name. **Input Ports (Edges):** Defined in parentheses (). Expressions here are algebraic edges, not strings (unless quoted). **Content Slot:** Defined in braces { }.

```
// Usage Example
\Component Flow(gap: Number = 16) {
  \Children {
    x: parent.left,
    y: prev ? prev.bottom + gap : parent.top
  }
}

// Applying the Flow
\Flow(gap: 24) {
  \Header(size: 32) { "Welcome to DAG-UI" }
  \Paragraph { "This layout is mathematically provable." }
}
```

5. Primitive Reference (The Engine ABI)

The language eventually compiles down to a flat list of these primitives for the Rust/Vello engine:

- `\Block`: A purely spatial construct. Evaluates layout math but passes zero draw commands to Vello.
- `\Rect`, `\Circle`, `\Path`: Paint primitives. They take spatial coordinates and color/stroke data and output Vello paths.
- `\Text`: The bridge to Parley.
 - **Input:** `string, max_width, font, size`.
 - **Output:** Passes data to Parley's line-breaker. Parley yields the `height` and exact glyph coordinates back to the graph.

6. Collision Resolution (Precedence)

If multiple edges attempt to drive the same input port, the compiler resolves them statically:

1. **Explicit Instance Wiring Wins:** `\Paragraph(x: 100)` overrides ambient layouts.
2. **Ambient Container Wiring:** `\Children { x: parent.left }` applies if the instance did not explicitly set `x`.
3. **Forced Container Wiring:** Using `x!: parent.left` in a `\Children` block throws a compiler error if a child attempts to override it.

How to direct your agent

When you pass this into your workspace/agent, give it a prompt like this to structure its work:

```
> *I am providing a specification document for a new functional reactive UI language called DAG-UI. We are bu
>
> *Your first task is to write the parser using `nom` or `logos`/`chumsky`. The parser must read this syntax a
This will force the agent to skip all the traditional web-framework boilerplate and start directly on your ma
```

User prompt: What's a good name for this markup format? Maybe something based on movable type? Or something related to the dag? Directed type? What do you think?

Response: **Directed Type** is actually fantastic. It sets up a perfect historical parallel: HTML stands for *HyperText* Markup Language because its core innovation was non-linear links *between* documents. Your language could be **DTML (Directed Type Markup Language)** because its core innovation is directed mathematical links *within* the document.

Here are a few other angles based on the DAG, math, and movable type concepts to see if anything sparks:

The "Movable Type" Angle

In traditional letterpress printing, physical blocks of type had to be locked together mathematically so they wouldn't fall apart in the press.

- **Chase (Chase Markup):** A "chase" is the heavy steel frame used to lock movable type into a rigid, mathematically perfect grid using pressure. It perfectly describes your engine: locking text and blocks into a provable, unmoving mathematical layout.
- **Matrix (Matrix Markup):** In type founding, the matrix is the physical mold used to cast the letterforms. It gives you a great double meaning—the typographic mold, and the algebraic matrix your engine uses to resolve the layout.
- **Forma / Form:** A "form" (or forme) is the completed, locked-up arrangement of type ready to be printed.

The DAG / Topological Angle

- **TopoType (Topological Type):** A nod to Kahn's Algorithm and the topological sort that powers your Phase 2 execution. It sounds highly technical and specific to your architecture.
- **Vertex / Vertex Markup:** In a DAG, you have edges and vertices. Since your markup defines the nodes (vertices) and the edges flow between them, Vertex is a clean, sharp name.
- **Kahn / KahnML:** Naming the language directly after Arthur Kahn (who published the topological sorting algorithm in 1962).

The Dependency & Wiring Angle

- **Bound / Bound Markup:** Emphasizes the "late-bound" algebraic edges that define the language.
- **Lattice:** A lattice is a partially ordered set in mathematics (closely related to DAGs), and visually represents an interwoven structure where everything relies on the specific position of everything else.
- **WireType:** Captures the essence of plugging input ports into output ports like a hardware synthesizer.

My Top 3:

1. **Directed Type (DTML):** It sounds like an industry standard waiting to happen. It directly contrasts with HyperText.
 2. **Chase:** Punchy, one syllable, deep roots in movable type, and perfectly describes "locking" a layout into place.
 3. **TopoType:** Playful but mathematically accurate.
-